

Begonnen am	Donnerstag, 23. Januar 2025, 17:01
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 12:07
Verbrauchte Zeit	19 Stunden 5 Minuten

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

Frage 1

Teilweise richtig

Nicht bewertet

Kontrollstrukturen

Implementieren Sie die hervorgehobene Codesequenz in strukturiertem Assembler, wie es der Compiler tut.

```
int32_t i;
int32_t a;
a = 5;
for (i = 10; i > 2; i--) {
    a += i;
}
```

Label	Assemblerdirektive
	MOVS R0,#10
	MOVS R1,#5
	B j_label
lbl_jp	SUBS R0,#1
	ADDS R1,R0,R1
j_label	CMP R0,#2
	BLT lbl_jp

BHI lbl_jp

TST R0,#2

BEQ lbl_jp

BLO lbl_jp

BCS lbl_jp

BGT lbl_jp

Die Antwort ist teilweise richtig.

	MOVS R0,#10
	MOVS R1,#5
	B j_label
lbl_jp	ADDS R1,R0,R1
	SUBS R0,#1
j_label	CMP R0,#2
	BGT lbl_jp

Frage 2

Richtig

Nicht bewertet

Gegeben ist folgendes Assembler-Codefragment:

```
...  
    MOVS    R1, #0  
...  
lbl_1  ...  
    ...  
    LDR     R1, =ADR_DIPSW  
    LDRB    R1, [R1]  
    MOVS    R0, #02  
lbl_2  TST    R1, R0  
    BEQ     lbl_1  
...
```

Um welche Kontrollstruktur handelt es sich hier?

☐ if ☐ for ☒ do-while ☐ while ☐ switch-case

Erreichte Punkte 1,00 von 1,00

Die richtige Antwort ist: do-while

Dies ist eine Do-While-Schleife; das Kennzeichen ist, dass der Schleifen-Test am Schluss ausgeführt wird. Oder anders ausgedrückt: Der Schleifen-Körper wird in allen Fällen mindestens einmal ausgeführt.

Der Code lädt ein Byte von der Adresse ADDR_DIPSW, d.h. die Stellung der DIP-Switches 7-0. Dann wird das Bit 1 getestet (Maske 0x02), und wenn es Null ist, dann wird zu lbl_1 zurück gesprungen.

Frage 3

Richtig

Nicht bewertet

Gegeben ist folgendes Assembler-Codefragment:

```
...  
    CMP     R2, #8  
    BLO     lbl_1  
...  
    B       lbl_2  
lbl_1  ...  
    ...  
lbl_2  ...
```

Um welche Kontrollstruktur handelt es sich hier?

☒ if ☐ for ☐ do-while ☐ while ☐ switch-case

Erreichte Punkte 1,00 von 1,00

Die richtige Antwort ist: if

Dies ist eine typische if-Anweisung mit den Blöcken else (bei lbl_1) und then (direkt nach BLO). Danach fährt der Code in beiden Fällen bei lbl_2 weiter.

Das Register R2 wird vorzeichenlos mit der Zahl 8 verglichen - wenn R2 grösser oder gleich 8 ist, wird der then-Code ausgeführt, ansonsten der else-Code (bei lbl_1).

Frage 4

Richtig

Nicht bewertet

Gegeben ist folgendes Assembler-Codefragment:

```
...  
MOVSB R1, #0  
B lbl_2  
lbl_1 ...  
...  
ADDS R1, #1  
lbl_2 CMP R1, R5  
BLE lbl_1  
...
```

Um welche Kontrollstruktur handelt es sich hier?

☐ if ☐ for ☐ do-while ☒ while ☐ switch-case

Erreichte Punkte 1,00 von 1,00

Die richtige Antwort ist: while

Dies ist eine While-Schleife, so, wie sie der C-Compiler generieren würde. Das Kennzeichen ist, dass nach der Initialisierung von R1 (erste Zeile) direkt zum Schleifentest gesprungen wird (Sprung auf Zeile 2, Schleifentest bei lbl_2). Im Gegensatz zu einer do-while-Schleife wird also der Schleifenkörper möglicherweise gar nie ausgeführt, weil der Test ja bereits beim ersten Mal fehlschlagen kann. Der Code initialisiert R1 mit 0, und testet, ob R1 kleiner oder gleich R5 ist (signed, also vorzeichenbehaftet). Wenn ja, wird der Schleifenbody bei lbl_1 ausgeführt, und vor dem nächsten Vergleich R1 um eins erhöht.



Datenschutz | Supportanfrage

[◀ Lab Control Structures](#)

Direkt zu:

[Lecture Subroutines and Stack ▶](#)[Barrierefreiheitserklärung ZHAW Moodle](#) | [Feedback zur ZHAW Moodle](#)[Barrierefreiheit](#)